```
164
165  func formatSolution(x float64) string {
166      if EqualFloat(x, 0, -1) {
167          return fmt.Sprintf("%.*f", decimals, x)
168      }
169      return fmt.Sprintf("%.*f", decimals, x)
170  }
```

The compilation process is simply as follows:

```
$ go build quad.go
```

The generated executable file must be running (using *./quad*) before trying to access the *http://localhost:8080* URL with your browser. The 8080 port is hard coded inside the Go code (line 44). Figure 1 shows two screens of the *quad* application.

The line-numbers are added in order to better refer to the Go code and need not be typed. I am now going to explain the code, line by line:

- Line 1-9: Comments about the Go program.
- Line 15: The *net/http* package uses the net package to add functionality for parsing HTTP requests and replies. It also provides a basic HTTP client as well as a basic HTTP server.
- Line 16: The *log* package offers logging functions.
- Line 17: The *math* package contains many math-related constants and functions.
- Line 18: The *strconv* package contains many functions for converting strings into other types and other types into strings. The *quad* program uses the *strconv.ParseFloat()* function that converts a string to a floating-point number.
- Line 21: At line 21, the definition of some string constants begins. Most of them are used for the HTML output. The *decimal* constant (Line 22) is used to define the number of digits after decimal points.
- Line 43: The *http.HandleFunc* function takes two arguments, a *path* and a reference to a *function* (*solveQuad*) that will be called when the path is requested. You can register as many pairs as you want.
- Line 44: The port number 8080 is defined in this line using the *http.ListenAndServe()* function. The *http.ListenAndServe()* function also starts the Web server.
- Line 45: If there is an error starting the Web server, an error message is displayed on the screen and the program exits. You will see an error example near the end of this article.
- Line 49: This line defines the *solveQuad()* function that does most of the job, as it is called whenever the website is visited. The *writer* argument is where the HTML response is written and the *request* argument holds the details of the HTTP request.
- Line 52: The *fmt.Fprint()* function writes its arguments to the *writer* using format *%v* and space-separated nonstrings, and returns the number of bytes written and an *error* or *nil*.
- Line 54: The *fmt.Fprintf()* function writes its arguments to the *writer* using the *format* string. It returns the number of bytes written, and an *error* or *nil*.
- Line 93: The *EqualFloat()* function compares two *float64* variables using the given accuracy. If a negative number is given as the accuracy level, the greatest possible accuracy will be tried. The function relies on both functions and constants from the *math* library package.
- Line 101: The *solve()* function finds the two solutions of the quadratic equation if they are real numbers.
- Line 102: *a*, *b* and *c* are read here so that the discriminant can be calculated.
- Line 104: If the value of the discriminant is less than 0, then the solutions are complex numbers, a scenario that is not supported by the *quad* program.
- Line 118: The *processRequest()* function reads the data in the Web form from the *request variable*. In case of an erroneous value, an error message is returned.
- Line 148: The *formatSolutions()* function takes two arguments and returns a string. It checks the number of discrete solutions and formats the solution or the solutions using the *formatSolution()* function.
- Line 165: The *formatSolution()* function formats a single value.

If the selected port is already in use, the error message you will get will be similar to the following:

```
2013/05/31 00:34:13 Exiting: Cannot start server: listen tcp
<nil>:8080: address already in use
```

I find it very impressive that you can write such an application in Go that also includes a Web interface, with just 170 lines of code, including the comments!

Summary

Go is a language that gets your work done. The easiest and quickest way to learn it is by programming and experimenting. This article is just the beginning of your journey to Go.

Until our next article, keep programming! END

Web links and bibliography

[1] Go: *http://golang.org*
[2] Go documentation: *http://golang.org/doc/*
[3] Go blog: *http://blog.golang.org*
[4] Go on Twitter: *https://twitter.com/go_nuts*
[5] Google App Engine: *https://developers.google.com/appengine/*
[6] *The Way To Go: A Thorough Introduction To The Go Programming Language*, Ivo Balbaert, iUniverse, ISBN: 1469769166, March 2012.
[7] You can find the source code at *http://www.linuxforu.com/article_source_code/Aug13/go_language.zip*.

By: Mihalis Tsoukalos

The author enjoys UNIX administration, programming iOS devices and photography. You can reach him at tsoukalos@sch.gr or @mactsouk.